Computer Animation - Crowd Simulation Project

Alexandra Maria Sicobean[1]

[1]Universitat Politècnica de Catalunya - BarcelonaTech

Nuria Pelechano Gomez, Alejandro Beacco Porres

1. Introduction

This project represents a small crowd simulation system developed through the Unity 3D Engine. The final scene displays multiple animated agents smoothly navigating a scene on updated paths, while avoiding obstacles and other agents.

2. Project Structure

2.1. Unity Project Organization

- **Scenes:** One scene per exercise
- **Scripts:** Found in the `Scripts` folder, shared across scenes
- **Prefabs:** Reusable agent and grid blocks
- **Assets:** Characters and animations

3. Exercises

3.1. Locomotion

Implemented Features:

1. Imported a rigged 3D character with multiple locomotion animations
2. Created an Animation Controller with a locomotion Blend Tree
3. Implemented a motion tracker
4. Introduced keyboard-driven movement for testing
5. Implemented automatic orientation based on velocity direction
6. Introduced fixed orientation through keyboard input (**F key**)

Relevant Scripts:

1. `Tracker`: Measures displacement and velocity
2. `LocomotionController`: Feeds the motion data to the `Animator`
3. `OrientationManager`: Smoothly rotates the character
4. `PlayerMovement`: Generates motion input (using the keyboard)

How to test the Scene:

Use the keyboard to move the character around. Press the **F key** to fix the character's orientation. Toggle the Gizmos and observe how direction changes (blue line). The green line represents velocity, which should be constant when the character is moving. When static, the character has a specific animation.

3.2. Collision Detection and Avoidance between agents

For this scene, I kept the agents simple (capsules) instead of the previously animated characters. But the final version contains the animated characters.

Implemented Features:

1. Instantiated a crowd of N agents at random, non-overlapping positions in a planar scene
2. Implemented a global `Simulator` that updates all agents every frame
3. Assigned random 2D goals to each agent
4. Detected when a goal is reached and reassigned a new random goal
5. Integrated collision detection and response using Unity's physics engine

Relevant Scripts:

1. `CrowdGenerator`: Spawns a crowd of agents with configurable size and spawn radius
2. `Agent`: Stores per-agent state such as velocity, goal, and speed
3. `Simulator`: Centralized update loop that moves agents and handles collision-aware motion

How to Test the Scene:

There is not much user interaction in this scene. Press play and observe how each agent moves towards a randomly assigned goal and selects a new one when it is reached. Capsule colliders and rigid bodies ensure that they do not overlap while in motion.

3.3. PathFinding

Implemented Features:

1. Generated a random regular grid with obstacles
2. Built a graph representation of the grid using custom node and connection classes
3. Implemented the A* pathfinding algorithm over the grid
4. Implemented a bidirectional A* variant to improve performance
5. Integrated pathfinding with the crowd simulation using waypoints
6. Extended the heuristic to penalize cells with high agent density

Relevant Scripts:

1. `Grid`: Generates the grid, obstacles, and graph connections
2. `GridCell`: Represents a navigation node with obstacle and density information
3. `CellConnection`: Represents valid movements between neighboring cells
4. `A_Star`: Generic implementation of the A* algorithm
5. `Bidirectional_A_Star`: Optimized A* variant using two simultaneous searches
6. `GridHeuristic`: Euclidean heuristic extended with agent-density penalties
7. `Agent`: Requests paths and follows grid-based waypoints

How to Test the Scene:

The last two scenes are merged into the same one. However, you can observe their different behavior through the right configurations. I allowed the inclusion of the `Simulator` in the main scene. You can change the mode during simulation:

Steering or simple Pathfinding (there is also collision detection, but it is used in another scene, not this one). To change between A^* and *bidirectional* A^*, you can use the configurations in the `GridManager`.

3.4. Steering Behaviors and Crowd Navigation

Implemented Features:

1. Steering-based path following using an *Arrive* behavior
2. Dynamic obstacle avoidance based on a forward look-ahead corridor
3. Inter-agent collision avoidance using velocity prediction
4. Density-aware Pathfinding through heuristic penalization
5. Runtime switching between A* and Bidirectional A* algorithms
6. Unified scene showcasing both steering and density effects

How to Test the Scene:

The testing is similar to the one in the previous section. Run the Main Scene. The default configuration will run with Steering instead of Pathfinding. You can modify the weight of each steering fore through the `Simulator`'s settings.